

35 — System Testing Specification

Version 1.0

Product-Level Engineering Specification

Testing rule: a feature is not complete when it works only on the happy path; correctness, safety, failure behavior, authorization, observability, and lifecycle effects must be demonstrated.

Document	Value
System	CloudShield AI
Specification	35 — System Testing
Version	1.0
Scope	Product-wide verification from unit level through end-to-end, security, resilience, release, and recovery testing
Primary Cloud	AWS for MVP
Depends On	00–34

Document Map

- 1. Purpose and Scope
- 2. Testing Objectives and Principles
- 3. System Under Test
- 4. Verification Strategy
- 5. Test Levels
- 6. Test Types
- 7. Test Environments
- 8. Test Data Strategy
- 9. Test Configuration and Isolation
- 10. Unit Testing
- 11. Component Testing
- 12. Contract Testing
- 13. Integration Testing
- 14. API Testing
- 15. Cloud Integration Testing
- 16. External Integration Testing
- 17. Database and Data Testing
- 18. Event and Async Testing
- 19. State and Lifecycle Testing
- 20. Identity and Dependency Testing
- 21. Security Testing
- 22. AI/LLM Testing
- 23. AI Safety and Guardrail Testing
- 24. DAIL Testing
- 25. Remediation and Promotion Testing
- 26. Evidence and Audit Testing

- 27. UI and UX Testing
- 28. End-to-End Testing
- 29. Performance and Scalability Testing
- 30. Reliability and Resilience Testing
- 31. Disaster Recovery Testing
- 32. Deployment and Release Testing
- 33. Observability Testing
- 34. Compatibility and Upgrade Testing
- 35. Regression Strategy
- 36. Exploratory and Manual Testing
- 37. Defect Management
- 38. Test Automation Architecture
- 39. CI/CD Test Gates
- 40. Test Metrics and Quality Criteria
- 41. Test Evidence and Reporting
- 42. MVP Test Baseline
- 43. Implementation Sequence
- 44. Security and Functional Acceptance Criteria
- 45. Traceability
- 46. Engineering Decisions
- 47. Definition of Done
- 48. Glossary

1. Purpose and Scope

This specification defines how CloudShield AI is tested as a complete product and how confidence is established before a change is released. It covers application behavior, domain state, cloud discovery and mutation, security analysis, dependency/impact, AI/LLM behavior, DAIL, evidence, APIs, asynchronous jobs, infrastructure, observability, performance, resilience, and disaster recovery.

The testing model follows the architecture and lifecycle principle that observed state, proposed changes, verification, governance, promotion, and trusted state are distinct. Tests therefore validate outputs as well as authorization boundaries and prohibited transitions.

1.1 In Scope

- Automated unit, component, contract, integration, end-to-end, security, performance, resilience, and recovery testing.
- Manual exploratory testing for high-risk workflows and UX behavior.
- Testing AWS adapters, Terraform, queues, databases, object storage, secrets, external integrations, and AI providers.
- Testing DAIL lifecycle, verification, impact, evidence, and promotion boundaries.
- Test environments, data, fixtures, mocks, simulators, test accounts, and evidence retention.
- CI/CD quality gates, release verification, regression management, defect classification, and test reporting.

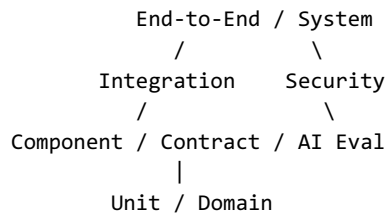
1.2 Out of Scope

- Replacing detailed implementation specifications for individual subsystems.
- Declaring the system secure solely from automated tests; security assurance also requires threat modeling, review, monitoring, and operations.
- Treating a passing suite as proof of every possible cloud or AI behavior.

2. Testing Objectives and Principles

Objective	Requirement	Expected Result
Correctness	Validate intended behavior and domain invariants.	Expected behavior demonstrated.
Safety	Verify dangerous or unauthorized paths are blocked.	Unsafe transitions fail closed.
Traceability	Tie tests to requirements, defects, releases, and artifacts.	Evidence is reconstructable.
Isolation	Prevent cross-tenant/environment/test contamination.	Boundaries remain intact.
Determinism	Prefer stable fixtures and controlled dependencies.	Failures are reproducible.
Realism	Use production-like staging and controlled cloud integration.	Important environmental behavior is exercised.
Resilience	Inject failures and validate recovery.	Partial failures have defined outcomes.
Regression Resistance	Repeat critical tests on material changes.	Known failures do not silently return.
Observability	Verify consequential actions are measurable/auditable.	Test outcomes include telemetry expectations.

2.1 Testing Pyramid



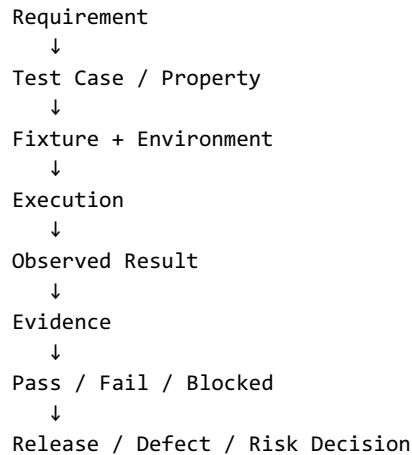
- Fast deterministic tests provide most feedback.
- Integration and system tests validate real boundaries.
- High-cost end-to-end tests target critical product workflows.
- Security, AI safety, DAIL, and lifecycle tests are first-class suites.

3. System Under Test

Area	Primary Behaviors Under Test
Product/API	Authentication, authorization, tenant isolation, commands, queries, errors.
Discovery	AWS connection, enumeration, pagination, normalization, reconciliation.
Security/Risk	Rule evaluation, findings, risk scoring, prioritization.
Dependency/Impact	Identity resolution, graph edges, blast radius, invalidation.
AI Intelligence	Context assembly, structured output, grounding, recommendation quality.
Remediation	Candidate generation, versioning, policy/lifecycle controls.
DAIL	Verification, impact, evidence, promotion gates.
Governance	Approval, rejection, policy evaluation, authorization.
Trusted State	Promotion, authoritative confirmation, drift/invalidation.
Evidence/Audit	Durable records, provenance, integrity, traceability.

Platform	Database, queue, object storage, secrets, observability.
Deployment	Artifacts, migrations, rollout, verification, rollback.
Integrations	AWS, identity, ITSM, source control, CI/CD, notifications, observability.

4. Verification Strategy



- Every critical requirement has an executable or reviewable verification method.
- State-changing tests verify preconditions, postconditions, and audit/evidence effects.
- Tests identify product, environment, dependency, test-infrastructure, or unknown failures.
- Unknown failures are not silently converted to passes.

5. Test Levels

Level	Purpose	Typical Runtime
Unit	Pure domain/function behavior.	Seconds.
Component	One service/module with controlled dependencies.	Seconds–minutes.
Contract	Compatibility across APIs/events/structured outputs.	Seconds–minutes.
Integration	Real database, queue, object store, provider sandbox/test account.	Minutes.
System	Multiple product components in realistic environment.	Minutes.
End-to-End	Golden-path user/product workflows.	Minutes.
Resilience/Recovery	Failure injection and restoration.	Minutes–hours.
Manual/Exploratory	Human discovery of unexpected behavior.	Variable.

6. Test Types

- Functional and negative-path testing.
- Boundary-value and property-based testing for domain calculations and invariants.
- Security testing: authentication, authorization, tenant isolation, injection, secrets, supply chain, abuse controls.

- AI quality and safety evaluation.
- Performance/load/stress/soak testing.
- Reliability, fault-injection, chaos-lite, and recovery testing.
- Migration/upgrade/compatibility testing.
- Accessibility and responsive UX testing.
- Deployment/rollback and infrastructure validation.
- Audit/evidence integrity testing.

7. Test Environments

Environment	Testing Role	Data Policy
Local	Unit/component, mocks, rapid feedback.	Synthetic only.
Test	Automated integration/system tests.	Synthetic or generated.
Staging	Release candidate, production-like validation.	Sanitized/non-sensitive test data.
AWS Integration Account	Controlled real-provider behavior.	Dedicated test resources and least privilege.
Production	Limited release verification and safe synthetic probes.	No destructive tests; approved checks only.
DR	Recovery and rebuild exercises.	Controlled recovery datasets.

- Cloud-mutating tests run only against explicitly designated scopes.
- Production test identities are read-only unless controlled mutation is explicitly required.
- Test resources use deterministic names/tags for cleanup and cost control.

8. Test Data Strategy

- Prefer synthetic, minimal, deterministic datasets.
- Cover healthy, stale, partial, conflicting, missing, malicious, and boundary conditions.
- Do not use sensitive production data as default fixtures.
- Cloud fixtures represent multiple regions/accounts/resource states and provider responses.
- AI datasets include benign, ambiguous, adversarial, injection, and policy-sensitive inputs.
- Expected results are versioned when rules/models/policies evolve.
- Cleanup is part of mutable-environment test lifecycle.

Dataset Class	Examples
Baseline	Normal assets, findings, candidates, dependencies.
Boundary	Empty scope, maximum pagination, large graph, long strings.
Failure	Timeout, throttling, permission denied, malformed provider response.
Security	Prompt injection, malicious external content, unauthorized tenant access.
Lifecycle	Every valid/invalid state transition.
Drift	Observed state differs from trusted state.
Reconciliation	Ambiguous or partial external mutation.
Recovery	Backup/restore and interrupted job state.

9. Test Configuration and Isolation

- Test configuration is explicit and environment-scoped.
- Mocks/fakes are distinguishable from real provider interactions.
- Parallel tests isolate tenants, scopes, and resource identifiers.
- Clock-dependent tests use controlled time where practical.
- Randomized tests record seeds for reproduction.
- Deterministic suites restrict uncontrolled network access.

10. Unit Testing

- Cover domain entities, lifecycle transitions, authorization predicates, risk calculations, normalization, and validation.
- Test valid and invalid state transitions.
- Use property-based tests for broad invariant spaces.
- Mock AWS, LLM, and external services at unit level.
- Keep unit tests fast enough for every pull request.

Candidate version V:
verify(V) succeeds only if V is immutable and eligible;
changing V after verification invalidates verification;
promotion without valid verification fails.

Unit tests prove local logic; they do not replace integration tests at trust boundaries.

11. Component Testing

- Exercise one deployable module/service with realistic internal dependencies.
- Validate configuration loading, authorization middleware, persistence adapters, serialization, and error mapping.
- Test startup/readiness behavior.
- Verify emitted events, audit records, and metrics where owned.

12. Contract Testing

Contract	Required Tests
REST API	Request/response schema, status codes, pagination, errors, compatibility.
Events	Schema compatibility, required fields, ordering assumptions.
AI Structured Output	Schema validation, enum constraints, refusal/uncertain states.
DAIL	Candidate/version/verification/evidence/promotion contract.
Cloud Adapter	Canonical domain mapping and error semantics.
External Connector	Webhook/event/OAuth capability contracts.

- Provider implementations satisfy provider-neutral contracts.
- Breaking contract changes require an explicit version or migration path.
- Consumer-driven contract tests are preferred for critical boundaries.

13. Integration Testing

- Use real persistence, queue, object storage, and controlled provider APIs where mocks are insufficient.

- Validate transaction boundaries and idempotency.
- Test eventual consistency and retry behavior.
- Use real scoped identities where practical.
- Capture integration evidence sufficient to reproduce failures.

14. API Testing

- Test authentication/authorization failures, tenant isolation, malformed requests, pagination, filtering, concurrency, idempotency, and rate limits.
- Verify clients cannot directly assign protected lifecycle states.
- Test optimistic concurrency/version conflicts.
- Validate stable error codes and safe error messages.
- Test async job acceptance and terminal status retrieval.
- Verify audit events for consequential commands.

15. Cloud Integration Testing

- Test AWS role assumption, discovery, pagination, throttling, timeouts, retries, and permission failures.
- Validate canonical resource identity and normalization.
- Test stale/partial responses and eventual consistency.
- Run controlled mutations only in dedicated integration scopes.
- Verify ambiguous mutation outcomes enter reconciliation.
- Test least privilege by deliberately denying unnecessary actions.

```
Cloud Test → AWS Read / Controlled Mutation
    → Observe Actual Resource
    → Normalize
    → Compare Expected vs Observed
    → Evidence
    → Cleanup / Reconcile
```

16. External Integration Testing

- Use provider sandboxes/mocks where available.
- Test OAuth scopes, token expiry, webhook signatures, retries, duplicates, and rate limits.
- Treat external content as untrusted input.
- Test connector outage without corrupting core state.
- Verify idempotent outbound delivery and durable retry state.

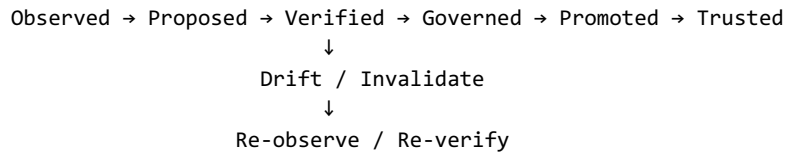
17. Database and Data Testing

- Validate schema constraints, indexes, transactions, uniqueness, foreign keys, and tenant scoping.
- Test migration forward paths and backward compatibility during rolling deployment.
- Test partial migration failure and restart.
- Verify audit/evidence records cannot be silently altered.
- Test backup/restore integrity.
- Validate retention and deletion behavior against policy.

18. Event and Async Testing

- Test enqueue/dequeue, retry, dead-letter behavior, visibility timeout, duplicate delivery, and idempotency.
- Test job cancellation and graceful worker shutdown.
- Test out-of-order events where permitted.
- Test stale job execution against newer candidate/context versions.
- Ensure stale workers cannot overwrite newer lifecycle state.
- Test queue backlog and worker scaling.

19. State and Lifecycle Testing



- Every permitted transition has positive tests and every forbidden transition has negative tests.
- Protected states require exact version/context binding.
- Verification cannot be reused after material candidate/context change.
- Approval is invalidated when protected inputs change.
- Promotion re-checks critical preconditions.
- Trusted state requires authoritative confirmation.
- Recovery tests validate reconciliation.

20. Identity and Dependency Testing

- Test deterministic asset identity across repeated discovery.
- Display-name changes must not create false identities.
- Ambiguous identifiers become conflicts rather than unsafe merges.
- Validate direct/transitive dependency traversal.
- Test missing, stale, low-confidence, and contradictory edges.
- Bind impact analysis to exact candidate version.
- Unknown blast radius cannot become a false bounded result.

21. Security Testing

Security Area	Minimum Coverage
Authentication	Invalid credentials, expiry, session/token boundaries.
Authorization	Role/scope checks, denied operations, privilege escalation.
Tenant Isolation	Cross-tenant read/write attempts.
Secrets	Leak prevention in source, logs, artifacts, errors.
Injection	SQL, command, template, API, webhook, prompt injection.
Cloud IAM	Least privilege and denied-action behavior.
Supply Chain	Dependencies, images, IaC, provenance.
Data Protection	Encryption, retention, secure transport.
Audit	Security events are durable and attributable.
Abuse	Rate limits, quotas, resource exhaustion controls.

Critical security failures block release.

22. AI / LLM Testing

- Test provider adapters independently from model behavior.
- Evaluate structured output validity, schema adherence, grounding, uncertainty, and recommendation relevance.
- Test prompt/context construction and data minimization.
- Test timeouts, malformed responses, quota errors, outages, and fallback behavior.
- Test routing changes against regression datasets.
- Do not treat fluent output as passing; domain validation remains mandatory.

Dimension	Example Check
Structure	Schema-valid response rate.
Grounding	Claims supported by authorized context.
Safety	Unsafe recommendations blocked.
Consistency	Controlled-case consistency.
Latency	p50/p95 response time.
Reliability	Successful structured-response rate.
Cost	Tokens/request within policy.
Injection Resistance	Malicious instructions do not override policy.
Abstention	Insufficient evidence is flagged appropriately.

23. AI Safety and Guardrail Testing

Untrusted Input
↓ Input Guardrail
↓ Authorized Context
↓ LLM
↓ Output Guardrail
↓ Domain Validation
↓ Tool/Action Boundary
↓ DAIL / Governance

- Test direct and indirect prompt injection.
- Test malicious instructions in resource names, tags, documents, tickets, code, and integrations.
- Test data-exfiltration attempts.
- Test tool calls with invalid, excessive, or unauthorized arguments.
- Test refusal/uncertainty when evidence is insufficient.
- Test kill switches, quotas, circuit breakers, and safe degradation.
- Prove AI cannot directly approve, verify, promote, or create trusted state.

24. DAIL Testing

- Test DAIL as a bounded subsystem integrated into CloudShield AI.
- Verify candidate immutability before verification.
- Bind verification to exact candidate version and context.
- Test dependency-aware impact and invalidation.
- Test verification failure, partial verification, timeout, and inconclusive results.
- Test evidence generation and linkage.

- Test promotion gates and restricted promotion identity.
- Test duplicate/concurrent verification requests.
- Test recovery and re-verification.

DAIL Property	Negative Test
Version Binding	Verify older version after candidate changes.
Immutability	Modify protected candidate after verification.
Safety Gate	Promote without valid verification.
Dependency Completeness	Remove critical edge and ensure safe handling.
Evidence	Complete verification without durable evidence.
Authorization	Invoke promotion with ordinary identity.
Concurrency	Conflicting verification/promotion operations.
Recovery	Restore stale verification and attempt promotion.

25. Remediation and Promotion Testing

- Test candidate creation from deterministic and AI-assisted recommendations.
- Verify immutable candidate versions after protected lifecycle points.
- Test impact analysis before verification.
- Test policy-based approval/rejection.
- Test promotion against current authoritative cloud state.
- Test mutation success, failure, timeout, and ambiguity.
- Verify promotion cannot bypass DAIL/governance.
- Verify successful promotion leads to authoritative observation and appropriate trusted-state transition.

26. Evidence and Audit Testing

- Critical operations create expected audit/evidence records.
- Verify actor, scope, version, policy, timestamps, and correlation identifiers.
- Test evidence integrity controls.
- Test retention and retrieval.
- Keep failed, rejected, rolled-back, and reconciled operations visible.
- Do not treat operational logs as authoritative proof.

27. UI and UX Testing

- Validate navigation, forms, filters, search, tables, graphs, dialogs, and responsive behavior.
- Test loading, empty, stale, partial, blocked, error, and permission-denied states.
- Verify observed/proposed/verified/trusted distinctions.
- Test destructive-action confirmation.
- Verify AI recommendations are non-authoritative until governed.
- Accessibility covers keyboard, labels, focus, contrast, and semantic structure.

28. End-to-End Testing

Connect AWS
→ Discover Assets

- Analyze Configuration
- Create Finding
- Assess Risk
- Assemble AI Context
- Generate Recommendation
- Create Candidate
- Analyze Impact
- DAIL Verify
- Approve
- Promote
- Observe AWS
- Confirm Trusted State
- Monitor Drift

- Maintain reliable golden-path tests.
- Add negative paths for denied promotion, stale context, verification failure, and mutation ambiguity.
- Validate evidence/audit side effects as well as visible outcomes.

29. Performance and Scalability Testing

Area	Test Focus
API	Latency, throughput, concurrent requests.
Discovery	Assets/regions/accounts and provider throttling.
Dependency Graph	Traversal and impact at expected graph sizes.
AI	Latency, concurrency, quota behavior, cost.
DAIL	Verification throughput and queue backlog.
Database	Latency, contention, connection limits.
Queue	Backlog recovery and worker scaling.
Evidence	Large artifact ingestion/retrieval.
UI	Initial load and large-result rendering.

- Use representative non-sensitive datasets.
- Define p50/p95/p99 targets for critical workflows before production hardening.
- Stress testing identifies graceful-degradation boundaries.

30. Reliability and Resilience Testing

- Inject database, queue, object storage, AWS, AI provider, external connector, and network failures.
- Test worker crash/restart and duplicate processing.
- Test dependency degradation and circuit breakers.
- Test partial deployment and mixed-version operation.
- Test stale state and reconciliation.
- Verify no failure path silently creates trusted state.

Fault → Detect → Contain → Persist State → Retry/Reconcile/Degrade → Verify → Recover → Evidence

31. Disaster Recovery Testing

- Execute database restore tests and verify compatibility.
- Rebuild infrastructure from Terraform in recovery environment.

- Deploy known-good immutable artifacts.
- Restore evidence/object storage and verify integrity.
- Recover queues/jobs and identify in-flight work.
- Test identity/secrets/KMS recovery.
- Test AI degradation during recovery.
- Test DAIL state recovery and re-verification.
- Reconcile authoritative cloud state before sensitive mutation.
- Conduct scheduled DR drills against Specification 33 targets.

32. Deployment and Release Testing

- Validate build provenance.
- Test artifact promotion across environments.
- Test database migration compatibility.
- Test rolling/canary/blue-green behavior where used.
- Test readiness and post-deploy smoke checks.
- Test rollback and forward-fix.
- Test partial deployment reconciliation.
- Verify deployment IDs link source/artifact/environment/actor/telemetry.
- Verify deployment cannot bypass product governance.

33. Observability Testing

- Verify required logs, metrics, traces, domain events, audit, and evidence.
- Verify correlation IDs propagate across API →job →AWS/AI/DAIL →persistence.
- Test secret/sensitive-data redaction.
- Test deployment, security, job, and trusted-state alerting.
- Test telemetry failure without unsafe state transitions.
- Verify critical audit events remain durable during telemetry degradation.

34. Compatibility and Upgrade Testing

- Test API consumers against supported versions.
- Test rolling deployment with old/new worker versions.
- Test event schema evolution.
- Test database compatibility during expand/contract migration.
- Test Terraform module upgrades before production.
- Test AWS SDK/provider adapter upgrades.
- Test LLM provider/model configuration changes.
- Test DAIL contract changes with producer/consumer compatibility.

35. Regression Strategy

Tier	When	Purpose
Tier 0	Every commit/PR.	Fast unit/static/contract checks.
Tier 1	Every PR.	Component/integration/security baseline.
Tier 2	Merge/main.	Broader system and E2E suite.

Tier 3	Release candidate.	Critical paths, performance subset, AI safety, DAIL, infrastructure.
Tier 4	Scheduled.	Deep resilience, DR, security, red-team, long-duration tests.

- Every production defect should yield a regression test when practical.
- Flaky tests are tracked as defects.
- Critical tests cannot be disabled without documented approval.

36. Exploratory and Manual Testing

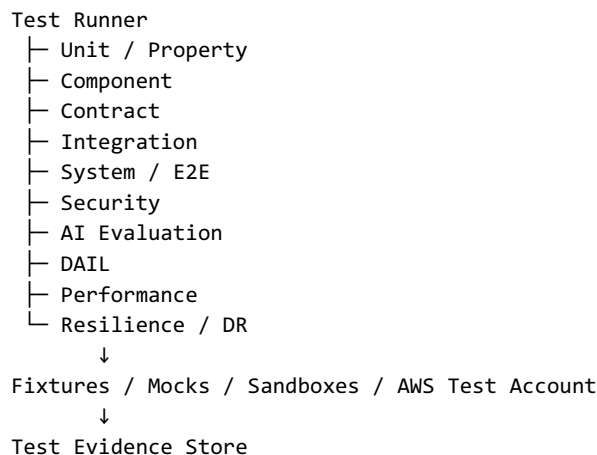
- Use exploratory sessions for new UX, complex graphs, AI explanations, degraded states, and emergent workflows.
- Challenge assumptions around stale data, permissions, conflicting dependencies, and partial cloud state.
- Convert repeatable findings into automated regression tests.
- High-impact changes receive targeted exploratory testing even with strong automation.

37. Defect Management

Severity	Meaning	Release Action
Blocker	Security, data-integrity, lifecycle, or critical availability failure.	Release blocked.
Critical	Major unsafe/incorrect behavior.	Normally blocked.
High	Important functional/reliability defect.	Fix or explicit risk acceptance.
Medium	Meaningful but bounded issue.	Track/schedule.
Low	Minor issue.	Backlog unless release-critical.

- Defects include reproduction, expected/actual result, environment, source/artifact, severity, and evidence.
- Security defects follow security workflow.
- State corruption or ambiguous cloud mutation defects require reconciliation analysis.

38. Test Automation Architecture



- Reusable fixtures cover tenants, assets, candidates, dependencies, policies, and cloud responses.
- Integration tests clean up mutable resources.

- Test utilities never provide production credentials.
- Test evidence is machine-readable where practical.

39. CI/CD Test Gates

Stage	Blocking Tests
PR	Lint, type/compile, unit, component, contract, baseline security.
Build	Dependency, secret, artifact/image, provenance.
Test	Integration and system tests.
Staging	Critical E2E, AI/DAIL, migration, security and smoke tests.
Production Preflight	Artifact integrity, configuration, migration readiness, approval.
Post-Deploy	Health, smoke, critical workflow, error-rate and security checks.
Progressive Rollout	Canary/rollout metrics and release verification.
Closure	Evidence/audit completeness and release record.

A gate is blocking when failure indicates continuing would violate a defined safety, correctness, or release policy.

40. Test Metrics and Quality Criteria

Metric	Purpose
Pass Rate	Basic suite health.
Flake Rate	Test reliability.
Defect Escape Rate	Pre-release effectiveness.
Critical Coverage	Coverage of high-risk requirements/workflows.
Mean Test Duration	Feedback efficiency.
Regression Detection	Ability to catch returned defects.
Security Finding Trend	Security quality over time.
AI Evaluation Score	AI quality/safety trend.
DAIL Gate Failure Rate	Lifecycle/verification stability.
Recovery Time	Operational readiness.

Code coverage is a supporting metric, not the definition of completeness. Risk-based coverage prioritizes authorization, lifecycle, cloud mutation, DAIL, AI safety, data integrity, and trusted-state transitions.

41. Test Evidence and Reporting

- Store test run ID, suite/version, source revision, artifact, environment, configuration version, timestamp, result, and failure details.
- Critical release reports link security scans, AI evaluations, DAIL tests, migration results, and deployment verification.
- Retain evidence according to audit/operations policy.
- Distinguish passed, failed, skipped, blocked, flaky, and not-applicable tests.
- Skipped critical tests require explicit justification.

42. MVP Test Baseline

- Unit tests for domain, lifecycle, authorization, identity, dependency, and risk logic.
- Component/contract tests for API, events, cloud adapters, AI structured outputs, and DAIL.
- Integration tests for PostgreSQL, queue, object storage, and controlled AWS discovery.
- Golden-path E2E from AWS connection through trusted-state workflow.
- Negative tests for unauthorized access, invalid transitions, stale context, and promotion bypass.
- Security baseline: SAST, dependency, secret, container, IaC, auth/authz, tenant isolation.
- AI baseline: structured output, grounding, injection, unsafe recommendation, provider outage.
- DAIL baseline: version binding, immutability, impact, verification, evidence, promotion.
- Performance baseline for critical APIs, workers, discovery, and AI.
- Resilience baseline for database/queue/AWS/LLM failures.
- Deployment/rollback smoke tests.
- Backup/restore and DR baseline.
- Observability/audit verification.

43. Implementation Sequence

1. Establish test framework, conventions, fixtures, and evidence schema.
2. Implement domain unit/property tests.
3. Implement API/component and authorization tests.
4. Implement contracts for APIs, events, AI, DAIL, and cloud adapters.
5. Implement persistence and queue integration tests.
6. Create controlled AWS integration account/scopes and harness.
7. Implement discovery/identity/dependency tests.
8. Implement security and tenant-isolation suites.
9. Implement AI quality and safety evaluations.
10. Implement DAIL lifecycle/verification/promotion suites.
11. Implement remediation/governance/trusted-state E2E tests.
12. Implement UI critical-path/accessibility tests.
13. Implement performance and resilience harnesses.
14. Implement deployment/rollback tests.
15. Implement backup/restore and DR tests.
16. Integrate gates into CI/CD and release reporting.
17. Establish scheduled deep regression, security, resilience, and DR programs.

44. Security and Functional Acceptance Criteria

ID	Acceptance Criterion
TEST-AC-001	Every critical requirement has an identified verification method.
TEST-AC-002	CI blocks releases on configured critical test/security failures.
TEST-AC-003	Tenant isolation is tested for reads and writes.
TEST-AC-004	Protected lifecycle states have positive and negative transition tests.
TEST-AC-005	Candidate verification is bound to exact candidate version/context.
TEST-AC-006	Promotion without DAIL/governance controls is proven to fail.
TEST-AC-007	AWS mutation ambiguity is tested and reconciled rather than treated as false success.

TEST-AC-008	AI prompt injection and unsafe tool-use paths are tested.
TEST-AC-009	AI cannot directly create verified, approved, promoted, or trusted state.
TEST-AC-010	Critical audit/evidence records are validated for durable creation.
TEST-AC-011	Database migration compatibility is tested across supported rollout sequences.
TEST-AC-012	Deployment rollback/forward-fix and partial-deployment reconciliation are tested.
TEST-AC-013	Critical failure modes are exercised through resilience tests.
TEST-AC-014	DR restore/rebuild is tested against defined recovery objectives.
TEST-AC-015	Release verification validates observed runtime health, not deployment success alone.

45. Traceability

Upstream Specification	Testing Dependency
00 — Master System	Product lifecycle, trust model, control-loop correctness.
01 — SRS	Functional, security, reliability, performance, data and acceptance requirements.
02 — Architecture	Service boundaries, trust boundaries, data flows, deployment topology.
03 — Functional	Use cases, transitions, async behavior, failure paths.
04 — Product & UX	UI states, workflows, accessibility, safe-path UX.
05 — Domain & Data	Entities, invariants, persistence, evidence/audit.
06 — State & Lifecycle	Allowed/forbidden transitions, version binding, reconciliation.
07 — Identity & Dependency	Stable identity, graph semantics, impact and invalidation.
08 — Security & Threat Model	Threat-driven security and abuse testing.
09 — Intelligence & AI Architecture	AI abstraction, context, routing, evaluation.
10 — LLM Integration	Provider contracts, errors, routing, quotas.
11 — AI Safety	Guardrails, injection, tool safety, degradation.
12–26 — DAIL Package	DAIL tests, verification, evidence, experiment, DevOps, DoD.
27 — AWS & Terraform	Infrastructure and AWS integration validation.
28 — API	API contract, authorization, lifecycle command restrictions.
29 — Cloud Integration	AWS discovery, mutation, reconciliation.
30 — External Integration	OAuth/webhook/connector reliability and security.
31 — Observability & Logging	Telemetry, correlation, audit/evidence verification.
32 — Security Operations	Detection, incident, security regression and response testing.
33 — Disaster Recovery	Restore, rebuild, failover and recovery validation.
34 — Deployment & DevOps	CI/CD gates, release verification, rollback, migration and deployment testing.

46. Engineering Decisions

- Testing is risk-based; correctness alone is insufficient for a cloud security control plane.
- Critical lifecycle and trust transitions require explicit negative testing.
- Real cloud integration is used where mocks cannot faithfully model provider behavior, only in controlled scopes.
- AI evaluation is a release input, not a one-time activity.
- DAIL testing is first-class and remains bounded within CloudShield AI.
- Production testing is safe, minimal, observable, and non-destructive by default.
- Flaky tests are engineering defects, not silently ignored noise.
- Test evidence supports release, audit, security, and incident investigation.
- DR and rollback testing are required because documented recovery without exercised recovery is insufficient assurance.

47. Definition of Done

- Critical requirements are mapped to tests or explicit verification methods.
- Unit/component/contract/integration suites are operational.
- API, tenant isolation, lifecycle, identity, dependency, and security tests are implemented.
- AWS/cloud integration tests run in controlled scopes.
- AI quality and safety evaluation suites gate relevant releases.
- DAIL verification, impact, evidence, and promotion tests are operational.
- Golden-path and critical negative E2E workflows pass.
- Database migration, deployment, rollback, and reconciliation tests are operational.
- Performance and resilience baselines exist.
- Backup/restore and DR tests have been exercised.
- Observability/audit/evidence assertions are included in consequential workflows.
- Test evidence is retained and linked to source/artifact/release.
- Critical defects block release unless governed by an approved risk decision.
- Test suites are integrated into Specification 34 CI/CD gates.
- Traceability to Specifications 00–34 is maintained.

48. Glossary

Term	Definition
SUT	System Under Test.
Unit Test	Test of a small isolated function or domain unit.
Component Test	Test of a module/service with controlled dependencies.
Contract Test	Test that verifies compatibility between system boundaries.
Integration Test	Test involving real infrastructure or external boundary behavior.
E2E	End-to-end test spanning a complete product workflow.
Regression Test	Test preventing a corrected behavior from returning.
Fixture	Controlled test data/configuration establishing known state.
Property Test	Test of an invariant across generated inputs.
Flaky Test	Nondeterministic test result without meaningful product change.
DAIL	CloudShield AI subsystem responsible for dependency-aware lifecycle, impact, verification and related controls.
AI Evaluation	Structured assessment of AI quality, safety, grounding, or

	reliability.
Test Evidence	Durable information demonstrating what was tested, how, and with what result.
Reconciliation	Resolution of differences between expected and authoritative observed state.
Trusted State	State earned only through required verification, governance, promotion, and authoritative confirmation.

Final testing rule: passing tests increase confidence; they do not grant authority. CloudShield AI must still enforce authorization, lifecycle gates, DAIL verification, governance, evidence, and authoritative cloud confirmation at runtime.